

Python Foundations — Speaker Notes

Interactive I/O, Arithmetic, and Real-World Logic

Total running time: about 28–30 minutes including questions. The notes follow the slide order. “What to say” is a suggested script you can paraphrase; “Ask the class” and “Watch out” are optional interaction points and common mistakes.

A suggested live-demo approach: have your IDE open next to the slides and run each code block (demo0-en.py to demo3-en.py) right after you explain it.

Slide 1 — Python Foundations (title)

🕒 1 min

What to say

Welcome the class and introduce today’s goal. Three ideas will carry the whole session: getting data from the user, doing math with it, and turning a real-world problem into code.

Explain that every example today follows the same pattern: Input → Processing → Output. If students remember only one thing, it should be this three-step structure.

Slide 2 — Input & Type Casting (section)

🕒 30 sec

What to say

We start with demo0-en.py. The key question for this section: what does Python do with whatever the user types?

Ask the class

- If a user types 25 on the keyboard, is that the number 25 or the text “25”? (Let them guess before the next slide.)

Slide 3 — Handling User Data

🕒 4–5 min

What to say

`input()` always returns a string, no matter what the user types. Even if they type 1998, Python receives the text “1998”, not the number.

To do math, we convert (cast) the string into an integer with `int()`. Point out the nesting: `int(input(...))` means ‘ask the user, then immediately convert the answer’.

Walk through the code in the same order as the comments: reading text, reading an integer, processing (subtracting the birth year), then output with `print()`.

Live demo

- Run the program and type your name and a birth year. Then run it again and type a word instead of a number for the year, so students see the `ValueError`.

Ask the class

- What would happen if we removed `int()` and tried `2026 - birth_year`? (`TypeError: you can’t subtract a string from an integer.`)

Watch out

- The result is only approximate (`est_age`) because we ignore the birth month and day. The slide says “approx.” for exactly this reason.

- The year 2026 is hard-coded. Mention that later in the course they'll learn to get the current year automatically.
- `print()` with commas inserts spaces between items automatically, which is why "Hello," and the name appear separated.

Slide 4 — Arithmetic Operators (section)

🕒 30 sec

What to say

Now to `demo1-en.py`. Python can do everything a calculator does, plus a few operators that are especially useful in programming.

Slide 5 — Beyond Basic Calculators

🕒 4 min

What to say

Left card: the standard operations. With `a = 15` and `b = 4` we get 19, 11, 60 and 3.75. The important point is that `/` always gives a float, even when the division is exact (for example `8 / 4` gives 2.0).

Right card: the three 'algorithmic' operators. `//` is integer (floor) division and gives the quotient: 3. `%` is modulo and gives the remainder: 3. `**` is the power operator: 15 to the power of 4 is 50625.

Tell students these appear constantly in programming: splitting values into parts, checking divisibility, cycling through lists, and so on.

Ask the class

- Before revealing the comments, ask what `a // b` and `a % b` would print. Many will guess 3.75 for `//`.

Watch out

- Students often confuse `**` with `^`. In Python, `^` is a different operator (bitwise XOR), not exponentiation.

Slide 6 — Understanding Modulo (%)

🕒 3 min

What to say

Modulo is the operator students find most confusing, so spend a little more time here. It does not divide; it returns what is left over after dividing.

Walk through the three bullets: 4 fits into 15 three times ($4 \times 3 = 12$); the difference between 15 and 12 is 3; so `15 % 4 = 3`.

Then explain why it matters: `n % 2 == 0` tells you whether `n` is even, and modulo keeps a number inside a fixed range (for example, an index that should wrap around a list).

Quick practice (30 seconds)

- Write on the board: `17 % 5`, `20 % 4`, `7 % 10`. Answers: 2, 0, 7.
- The last one surprises people: when the first number is smaller than the second, the result is the first number itself.

Watch out

- Don't forget to point out the difference between `==` (comparison) in `n % 2 == 0` and `=` (assignment). It will come up again when you introduce if statements.

Slide 7 — Practical Geometry (section)

🕒 30 sec

What to say

Moving to demo2-en.py: applying what we know to a classic problem, the area and perimeter of a rectangle.

Slide 8 — Area and Perimeter Logic

🕒 4 min

What to say

Here we apply the Input → Processing → Output structure in full. Read two numbers (length and width), compute two results, and print them.

Formulas: $\text{area} = \text{length} \times \text{width}$, $\text{perimeter} = 2 \times (\text{length} + \text{width})$. Stress the parentheses: without them, $2 * L + l$ would give the wrong perimeter.

Point out how descriptive variable names and comments labelling each stage make the program easy to read.

Note about this slide

- The code on this slide uses Romanian text and variable names (aria, perimetrul, “Introdu lungimea...”), while the rest of the deck is in English. If your class is in English, you may want to explain that it’s the same logic in Romanian: aria = area, perimetrul = perimeter, lungimea = length, lățimea = width.
- The slide uses L for length and l (lowercase L) for width. In many fonts, l looks like 1 or I, so consider telling students that in their own programs they should use clearer names such as length and width.

Live demo

- Try 5 and 3: the area is 15 and the perimeter is 16.

Ask the class

- Why do we use `int()` here? What if the user enters 2.5? (They would get a `ValueError`. This is a good lead-in to `float()` on slide 10.)

Slide 9 — The Taxi Fare Problem

🕒 2 min

What to say

Now a more realistic problem. A taxi charges a flat base fare plus a rate per mile. Normally we’d compute the total from the miles; this time we know the total and must work backwards to find the miles.

Say that real programming often means translating a word problem into math first, and only then into code.

Ask the class

- Before moving on, ask: what are the three values we need, and what is the one we’re solving for? (Inputs: base fare, rate per mile, total cost; output: miles.)

Slide 10 — Isolating Variables in Code

🕒 5 min

What to say

Start with the math on the left. The forward formula is $\text{total} = \text{base} + \text{rate} \times \text{miles}$. To isolate miles, first subtract the base fare from the total (this gives the cost of the distance alone), then divide by the rate per mile.

Then the implementation on the right, again with the three stages: Input, Processing, Output.

Explain why we use `float()` here and not `int()`: money has decimals (for example, \$3.50).

Worked example for the board

- Base fare \$3.50, rate \$2.00 per mile, total \$13.50.
- $\text{dist_cost} = 13.50 - 3.50 = 10.00$
- $\text{miles} = 10.00 / 2.00 = 5.0$

Watch out

- Beware of division by zero if the user enters a rate of 0 (`ZeroDivisionError`). You don't need to solve this today, but you can mention that checking for such cases will come with `if` statements.
- If total is smaller than base, the result is negative miles, which makes no sense in real life.
- Some inputs may print values like 4.999999999 because of floating-point precision. If it happens, it's a nice opportunity for a short explanation (and for `round()` later).

Slide 11 — End of Demos

 3 min (questions)

What to say

Recap the three ideas: `input()` returns text, so we cast it with `int()` or `float()`; Python's operators include `//`, `%` and `**`; and every program can be structured as Input → Processing → Output.

Invite questions about logic extraction, casting or operators, then encourage the class to keep practicing in their IDE.

Suggested practice tasks

- Convert minutes to hours and minutes using `//` and `%`.
- Compute the area of a circle (use 3.14 for pi, and `**` for the square).
- Reverse the taxi problem: given miles, rate and base, compute the total cost.
- Check whether a number typed by the user is even or odd, using `%`.

Slide 12 — Image Sources

 No presenting time

What to say

Credits slide. You don't need to present it; leave it as a reference. It lists the sources of the three images (coding photo and taxi photo from stockcake.com, and the geometric pattern from redbubble.com).